

Contents

The bootcamp at a glance	3
Week 1 — Building AI Applications	3
Theme	3
Day 1 — The new AI application stack	3
Day 2 — Context engineering	4
Exercise	5
Day 3 — RAG	5
Day 4 — Evals	6
Offline evaluation	6
Metrics	6
Day 5 — Agentic workflows	7
Day 6 — Production AI	8
Day 7 — Week 1 project	9
Personal Research Assistant	9
Week 2 — Software Engineering for the AI Era	9
Day 8 — Architecture	10
Day 9 — Data systems	10
Why choose one over another?	10
Day 10 — Reliability engineering	11
Day 11 — Security	11
Agent security	12
Day 12 — Performance and economics	12
Day 13 — Testing AI systems	13
AI regression suite	13
Day 14 — Architecture review	13
Week 3 — Coding Agents	14
Day 15 — How coding agents work	14
Day 16 — Specification engineering	15
Day 17 — Agent context management	16
Day 18 — Agentic development loops	16
Day 19 — Multi-agent systems	17
Parallel	17
Pipeline	17
Manager/worker	17
Critic	17
Day 20 — Coding-agent safety	18
Day 21 — Agentic software project	18
Week 4 — Shaping the Build	19
Day 22 — Product thinking	19
Day 23 — AI-native product design	19

What becomes possible because intelligence is cheap?	20
Day 24 — MVP design	20
Day 25 — Build	20
Day 26 — User testing	20
Day 27 — Production hardening	21
Day 28 — Final evaluation	21
Technical evaluation	21
AI evaluation	22
Product evaluation	22
Day 29 — Architecture/product review	22
1. Problem	22
2. Product	22
3. Architecture	22
4. AI system	22
5. Evaluation	22
6. Economics	22
7. Failures	22
8. Roadmap	23
Day 30 — The AI Engineer’s future	23
What has changed?	23
The continuous thread: one project	24
Project: AI Research & Engineering Agent	24
The technical stack I’d use	25
Language	25
Models	25
AI infrastructure	25
Application infrastructure	25
Agent tooling	26
What I would deliberately NOT teach	26
The assessment system	26
The deeper curriculum I’d build for you	27
Track A — AI systems	27
Track B — Agent systems	27
Track C — AI economics	28
And there is one particularly important idea I’d make the centerpiece	28
Yes. I would not make this a conventional “learn AI” bootcamp. The post is really describing a transition in the engineering discipline:	
From writing code → specifying systems → supervising agents → evaluating probabilistic behavior → shaping products.	

Given your technical background, I'd make this a **4-week, project-heavy AI Engineering Deep Dive**, with substantially more depth than a typical DeepLearning.AI-style introductory course.

The bootcamp at a glance

- **Duration:** 4 weeks
- **Time:** ~25–30 hours/week
- **Format:** 20% theory, 60% building, 20% analysis/review
- **Capstone:** Build and deploy a production-quality AI application using coding agents, RAG/context engineering, agentic workflows, evals, observability, and a real deployment stack.

The four weeks map directly onto Ng's four skills:

Week	Core skill	Central question
1	Building & deploying AI applications	How do you build reliable software around probabilistic models?
2	Software engineering fundamentals	How do you architect the system and make the right tradeoffs?
3	Using coding agents	How does the engineer become an effective supervisor/orchestrator of AI coding agents?
4	Shaping the build	What should we build, why, and how do we turn an idea into a robust product?

But I would thread **evals + context engineering + agentic workflows + coding agents** through all four weeks rather than treating them as isolated topics.

Week 1 — Building AI Applications

Theme

LLMs are probabilistic components. Engineering means turning them into reliable systems.

The first week should establish the conceptual foundation.

Day 1 — The new AI application stack

Topics:

- Foundation models

- APIs vs local models
- Tokens and context windows
- Prompting
- Structured outputs
- Tool calling
- Multimodal models
- Inference parameters
- Model selection
- Latency/cost/quality tradeoffs

Build:

Mini-project #1: Model playground

Build a Python application that can:

- call multiple models
- stream responses
- measure latency
- count tokens
- estimate cost
- compare outputs
- enforce structured JSON output

The point isn't the application itself. It's learning to treat an LLM as an **engineering component**.

Day 2 — Context engineering

This deserves substantially more attention than prompting.

Study:

- system/user/tool context
- context windows
- instruction hierarchy
- context compression
- retrieval
- long-context failure modes
- relevance vs completeness
- context pollution
- state vs context
- memory

Then implement:

User

↓

Intent analysis

↓
Context retrieval
↓
Context construction
↓
LLM
↓
Structured output

Exercise

Give the system 100 documents and ask increasingly difficult questions.

Measure:

- retrieval precision
- retrieval recall
- answer accuracy
- hallucination rate

This naturally introduces **eval-driven development**.

Day 3 — RAG

Go deeper than “embed documents and search a vector database.”

Study:

- embeddings
- semantic search
- chunking
- metadata
- hybrid retrieval
- reranking
- query expansion
- multi-query retrieval
- contextual retrieval
- citation generation

Build a RAG system.

Then deliberately break it.

Examples:

- bad chunk boundaries
- irrelevant documents
- conflicting documents
- outdated documents

- adversarial documents
- questions requiring multiple documents

The objective is to learn:

RAG isn't a feature. It's a probabilistic information-retrieval system that requires measurement.

Day 4 — Evals

This is probably the most important day of Week 1.

Study:

Offline evaluation

- golden datasets
- deterministic tests
- LLM-as-judge
- human evaluation
- pairwise evaluation
- rubric-based evaluation

Metrics

Depending on the application:

- accuracy
- precision/recall
- groundedness
- relevance
- completeness
- hallucination rate
- tool-call success
- latency
- cost

Build an evaluation harness:

```
Dataset
↓
Application
↓
Outputs
↓
Evaluator
↓
Metrics
```

↓
Regression report

Then modify your application and watch the evaluation suite catch regressions.

This is where the traditional software-engineering mindset starts merging with AI engineering.

Day 5 — Agentic workflows

Study the progression:

Prompt

↓

LLM

→

Prompt

↓

LLM

↓

Tool

→

Planner

↓

Tool

↓

Observation

↓

Reasoning

↓

Tool

↓

...

Topics:

- agents vs workflows
- tool calling
- planning
- state
- loops
- retries
- reflection
- delegation
- stopping conditions

- permissions
- failure recovery

Build a small agent that can:

- search
- retrieve information
- call tools
- reason over results
- produce a final report

Then introduce failures deliberately.

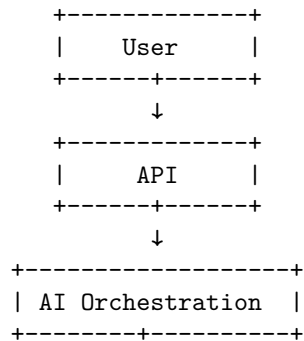
Day 6 — Production AI

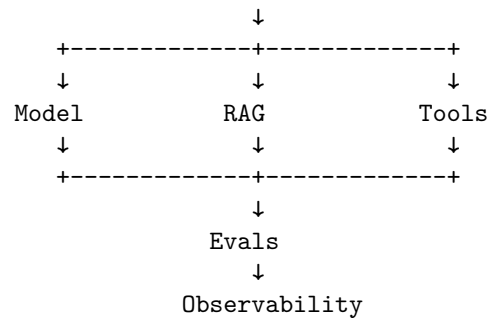
Now move from prototype to engineering.

Study:

- API architecture
- authentication
- rate limiting
- caching
- queues
- retries
- observability
- logging
- tracing
- secrets
- privacy
- security
- prompt injection
- data exfiltration
- cost controls

Architecture exercise:





Day 7 — Week 1 project

Build a complete AI application.

I would make the assignment:

Personal Research Assistant

It should:

- ingest documents
- retrieve relevant information
- answer questions
- cite sources
- use tools
- maintain conversational state
- detect uncertainty
- produce structured outputs
- have an evaluation suite
- expose metrics

And then deploy it.

Deliverable: working application + architecture diagram + evaluation report.

Week 2 — Software Engineering for the AI Era

This week is deliberately **not primarily about AI**.

That's important.

Ng's second point is that AI makes software engineering fundamentals **more valuable**, not less.

Day 8 — Architecture

Study:

- modularity
- interfaces
- coupling/cohesion
- APIs
- service boundaries
- dependency inversion
- event-driven architecture
- state management

Take your Week 1 application and redesign it.

Ask:

What happens if this application gets 1,000 users?

Then:

What happens at 1 million?

Day 9 — Data systems

Study:

- relational databases
- document databases
- vector databases
- caches
- object storage
- queues
- event streams

Most importantly:

Why choose one over another?

Have the agent propose an architecture.

Then **you critique the agent's architecture.**

This is an important recurring exercise:

Let AI design it. You identify the bad assumptions.

Day 10 — Reliability engineering

Study:

- failure domains
- retries
- exponential backoff
- idempotency
- circuit breakers
- graceful degradation
- timeouts
- load shedding
- SLOs/SLIs
- disaster recovery

AI-specific reliability:

- model unavailable
- model behavior changes
- malformed output
- hallucinations
- tool failure
- retrieval failure
- context overflow
- runaway agent

Design:

“What happens if every component fails?”

Day 11 — Security

Deep dive:

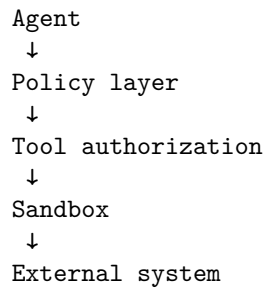
- authentication
- authorization
- secrets
- least privilege
- sandboxing
- prompt injection
- indirect prompt injection
- tool poisoning
- data leakage
- supply-chain attacks
- model output validation

Particularly:

Agent security

Never give an agent unrestricted capabilities.

Design:



Then try attacking your own system.

Day 12 — Performance and economics

This should be particularly interesting given your systems background.

Study:

- latency
- throughput
- batching
- caching
- concurrency
- context length
- model routing
- quantization
- inference cost
- GPU utilization

Build a simple cost model:

$$C = N_{requests} \times (T_{input}P_{input} + T_{output}P_{output})$$

Then optimize it.

For example:

Model A

- cheap
- slow
- high quality

Model B

- expensive
- fast
- high quality

Design a routing strategy.

Day 13 — Testing AI systems

Traditional testing:

input → deterministic output

AI testing:

input → distribution of possible outputs

Study:

- unit tests
- integration tests
- property-based tests
- regression tests
- evals
- adversarial tests
- fuzzing
- red teaming

Create:

AI regression suite

Every change to the system automatically runs:

100 test cases

↓

quality

latency

cost

safety

tool accuracy

↓

PASS / FAIL

Day 14 — Architecture review

Take the Week 1 application.

Perform a formal architecture review.

Produce:

1. architecture diagram
2. API specification
3. data model
4. threat model
5. cost model
6. reliability model
7. evaluation strategy
8. performance model

This is where you transition from **developer** to **AI systems engineer**.

Week 3 — Coding Agents

This is the most transformational part of the curriculum.

The goal isn't:

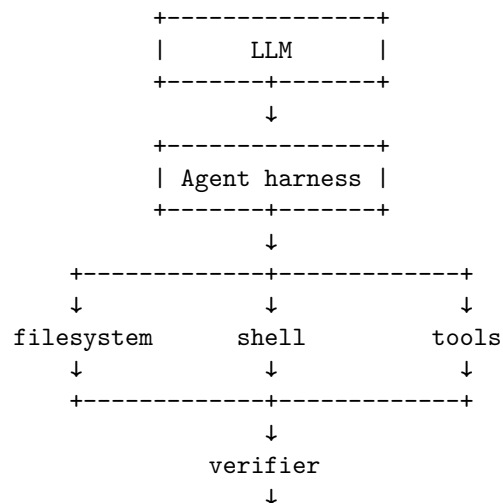
“Learn how to use Cursor/Claude Code/etc.”

The goal is:

Understand how to program an AI software engineer.

Day 15 — How coding agents work

Study the architecture of a coding agent:



feedback

Study:

- context management
 - tool use
 - planning
 - execution
 - verification
 - iteration
 - compaction
 - subagents
 - permissions
-

Day 16 — Specification engineering

This is a huge skill.

Compare:

“Build me a RAG application.”

with:

“Build a RAG service with these interfaces, constraints, tests, latency targets, security requirements, and acceptance criteria.”

Study:

- requirements
- invariants
- acceptance criteria
- interfaces
- constraints
- test cases
- architecture decision records

Exercise:

Give the same problem to an agent using:

Prompt A: vague specification

Prompt B: precise specification

Measure the difference.

Day 17 — Agent context management

This is one of the most important emerging skills.

Study:

- context budgeting
- context selection
- context compression
- summaries
- state
- scratchpads
- repository maps
- documentation
- task decomposition

Experiment:

Give the agent:

- entire repository
- relevant files only
- architecture summary
- tests
- explicit constraints

Measure performance.

You will see why **context engineering is becoming a software-engineering discipline.**

Day 18 — Agentic development loops

Learn:

SPEC

↓

PLAN

↓

IMPLEMENT

↓

TEST

↓

VERIFY

↓

FIX

↓

RETEST

Then build an autonomous loop.

The key concept:

Don't make the model smarter. Make the feedback loop better.

Examples of verifiers:

- compiler
 - unit tests
 - static analyzer
 - benchmark
 - evaluator
 - simulator
 - type checker
 - browser test
 - security scanner
-

Day 19 — Multi-agent systems

Study when multiple agents actually make sense.

Patterns:

Parallel

Task $\rightarrow$ $\leftrightarrow$ Agent A
 $\leftrightarrow$ Agent B
 $\leftrightarrow$ Agent C

Pipeline

A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D

Manager/worker

Manager
/ \
Worker Worker

Critic

Developer $\rightarrow$ Critic $\rightarrow$ Developer

Then build one.

But critically ask:

Does adding agents actually improve the system?

Agent multiplication is often cargo cult.

Day 20 — Coding-agent safety

Give your agent increasingly dangerous capabilities.

Examples:

- filesystem
- shell
- network
- database
- cloud deployment

Learn:

- sandboxing
- permissions
- allowlists
- human approval
- rollback
- transaction boundaries
- secrets isolation

Exercise:

Design an agent that **cannot destroy production** even if the model behaves maliciously or incorrectly.

Day 21 — Agentic software project

Take your Week 1 application and have the coding agent:

- redesign it
- add features
- write tests
- benchmark it
- fix bugs
- improve documentation

Your role:

You do not write most of the code.

You:

- specify

- review
- test
- evaluate
- redirect
- architect

This is the closest part of the bootcamp to the future engineering workflow Ng is describing.

Week 4 — Shaping the Build

This is the part most technical AI courses neglect.

The question becomes:

What should we build?

Not:

“How do we implement this?”

Day 22 — Product thinking

Study:

- user problems
- jobs-to-be-done
- user interviews
- pain points
- workflows
- willingness to pay
- adoption friction
- competitive advantage

Exercise:

Identify **10 problems** that could plausibly benefit from AI.

Rank them according to:

Opportunity = Pain × Frequency × AI leverage

Day 23 — AI-native product design

Ask:

What becomes possible because intelligence is cheap?

Examples:

- natural-language interfaces
- personalized workflows
- autonomous research
- continuous monitoring
- adaptive software
- natural-language programming
- multimodal interaction

Then ask the more important question:

What products were impossible before AI?

Day 24 — MVP design

Take one idea.

Create:

- user persona
- problem statement
- workflow
- product requirements
- architecture
- success metrics
- evaluation metrics
- MVP scope

Then give the specification to a coding agent.

Day 25 — Build

This is a full build day.

The agent does most implementation.

You act as:

Product owner + architect + evaluator.

Day 26 — User testing

Put the product in front of users.

Observe:

- where they get confused
- what they don't trust
- where the AI fails
- where latency matters
- where users want control
- what they actually value

Then revise the product.

This teaches one of the most important AI-engineering habits:

Don't spend three weeks polishing something users don't want.

Day 27 — Production hardening

Take the prototype and make it real.

Add:

- authentication
 - monitoring
 - logging
 - evals
 - security
 - cost controls
 - error handling
 - rate limits
 - deployment
 - documentation
-

Day 28 — Final evaluation

Run:

Technical evaluation

- functionality
- reliability
- latency
- cost
- scalability
- security

AI evaluation

- accuracy
- hallucination
- groundedness
- robustness
- tool-use success

Product evaluation

- user value
 - usability
 - adoption
 - differentiation
-

Day 29 — Architecture/product review

Give a 30-minute presentation:

1. Problem

What are we solving?

2. Product

What did we build?

3. Architecture

How does it work?

4. AI system

Why did we use AI here?

5. Evaluation

How do we know it works?

6. Economics

What does it cost?

7. Failures

Where does it fail?

8. Roadmap

What would we build next?

Day 30 — The AI Engineer's future

The final day should be synthesis rather than another technology lesson.

Discuss:

What has changed?

2022:

Requirements

↓

Architecture

↓

Code

↓

Tests

↓

Deployment

2026:

Problem

↓

Specification

↓

Context

↓

Agent(s)

↓

Generated implementation

↓

Automated verification

↓

Evaluation

↓

Human judgment

And potentially:

Human

↓

Intent

↓
Specification
↓
Agent swarm
↓
Software
↓
Automated evaluation
↓
Deployment

The engineer increasingly becomes the **designer of the system that produces software**, rather than the person who manually produces every line of software.

The continuous thread: one project

I would strongly recommend **one evolving project throughout the month**, rather than 30 disconnected tutorials.

For example:

Project: AI Research & Engineering Agent

Week 1:

Make it work

→ LLM + RAG + tools + evals

Week 2:

Make it robust

→ architecture + security + reliability + performance

Week 3:

Make the agent build it

→ coding agents + specs + verification + multi-agent workflows

Week 4:

Make it valuable

→ product definition + users + MVP + deployment

By Day 30 you have something substantially more valuable than a collection of notebooks.

The technical stack I'd use

I would deliberately keep the stack relatively narrow.

Language

Python + TypeScript

Python for:

- AI/ML
- evaluation
- data
- experimentation

TypeScript for:

- application layer
- APIs
- frontend

Models

Use multiple model providers rather than becoming dependent on one.

Learn the **model abstraction**, not just one API.

AI infrastructure

Learn:

- model APIs
- embeddings
- vector search
- structured outputs
- tool calling
- RAG
- agent frameworks

But don't hide the underlying mechanics behind frameworks.

Application infrastructure

- Git
- Docker
- PostgreSQL
- Redis
- object storage

- REST APIs
- queues
- cloud deployment

Agent tooling

Use **at least two different coding agents** and compare their workflows.

The objective isn't learning product-specific keystrokes. It's learning:

context → specification → planning → execution → verification → iteration.

What I would deliberately NOT teach

This is important.

A one-month bootcamp can easily become an enormous catalog of AI technologies.

I would **not** spend significant time on:

- training an LLM from scratch
- transformer implementation from scratch
- mathematical derivation of attention
- obscure agent frameworks
- endless prompt-engineering tricks
- fine-tuning unless the project requires it
- memorizing APIs
- chasing every new model
- building toy chatbots
- “100 LLM projects”

Those are interesting, but they don't map strongly onto Ng's four skills.

The assessment system

I'd make the bootcamp unusually rigorous.

Every project gets scored on five dimensions:

Dimension	Weight
AI system quality	20%
Software engineering	20%
Agent utilization	20%

Dimension	Weight
Evaluation/reliability	20%
Product judgment	20%

And I'd require **evidence**, not claims.

For example:

“Our RAG system is good.”

is meaningless.

Instead:

“On a 500-question evaluation set, retrieval recall@5 increased from 71% to 89%, grounded-answer accuracy increased from 76% to 91%, while p95 latency increased by 140 ms.”

That is AI engineering.

The deeper curriculum I'd build for you

Given your existing systems/ML background, I would actually make **your version considerably more advanced** than this baseline.

I would add three tracks running underneath the four weeks:

Track A — AI systems

Go deeper into:

- inference
- KV cache
- batching
- quantization
- model routing
- GPU/ANE/NPU utilization
- distributed inference
- latency/throughput
- memory bandwidth

Track B — Agent systems

Go deeper into:

- agent harness architecture
- context management

- planning
- tool protocols
- memory
- subagents
- verifiers
- agent evaluation
- autonomous loops
- multi-agent coordination

Track C — AI economics

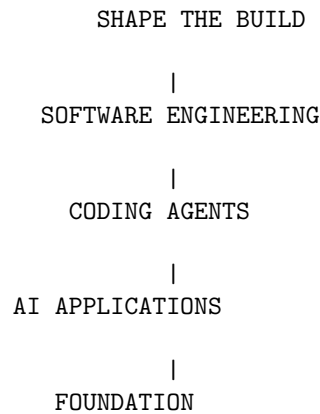
Study:

$$\text{AI value} = \frac{\text{capability} \times \text{adoption}}{\text{latency} \times \text{cost} \times \text{failure rate}}$$

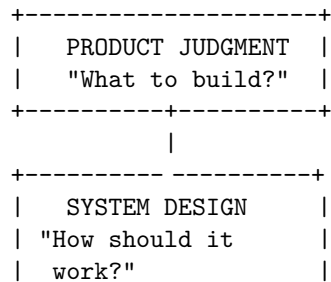
This connects technical architecture to the business/product decision.

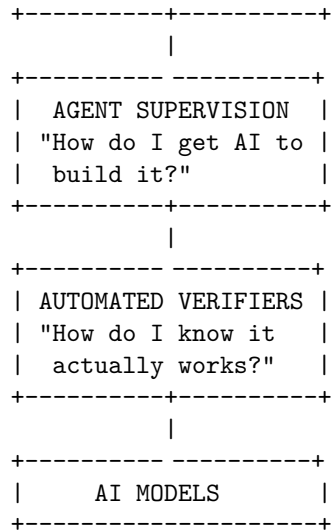
And there is one particularly important idea I'd make the centerpiece

The post's four categories can actually be arranged as a **hierarchy**:



But I think the real future hierarchy is closer to:





That is the curriculum I'd optimize for.

And given the kinds of questions you've been asking me recently—agentic harnesses, test-time training, scaling laws, AI accelerators, ML systems, and the changing role of software engineering—I would push this beyond a generic AI-engineering bootcamp into something closer to **“AI Systems Engineering 2026: From LLM Applications to Agentic Software Engineering.”** That would make the month much more intellectually demanding and much more aligned with where your existing expertise gives you an advantage.